Observability:

Observability is the **ability to understand a system's internal state** by examining its external outputs, like **logs, metrics, and traces**.

It allows you to **diagnose issues, understand performance, and improve reliability** by analysing data generated by the system.

Essentially, it's about being able to answer "why" something is happening within a system, even if you didn't anticipate the problem.

What is LangSmith

LangSmith is a unified **observability & evaluation platform** where teams can **debug, test, and monitor AI app** performance.

What does LangSmith Trace?

1. Input and Output
2. All the intermediate steps and its input output
3. Latency (Time taken to execute)
4. Token usage
5. Cost
6. Error
7. Tags
8. Metadata
9. Feedback

There are three major components in LangSmith

- **Project** – A complete setup or complete application is known as project
- **Trace** – A single execution of a flow is known as Trace, if program run 5 times, then we can say its 5 Trace.
- **Run** – Components in the single trace is called run, e.g. Prompt, Model, Parser in single trace count as separate run

RAG Failure Modes & LangSmith Recording

RAG apps have two big failure modes:

1. Retriever errors – wrong / irrelevant docs retrieved.
2. Generator errors – model hallucinates or misuses context.

In production, it's often unclear where the failure happened. Was the retriever bad, or did the LLM ignore the docs?

LangSmith automatically records:

1. User query
2. Retrieved documents
3. LLM prompt (with inserted docs)

4. LLM response

For RAG Applications

It triggers the build path in exactly these cases:

- No cache exists yet means First-ever run
- PDF content changes.
- PDF file metadata (for e.g. size) changes.
- Chunking params change –> chunk size or chunk overlap.
- Embedding model name changes –> e.g., from "text-embedding-3-small" to another model.

When using with LangGraph

1. **Every graph execution** can be logged in LangSmith as a **trace**.
2. **Each node** (e.g., retriever, LLM, tool call, subgraph) becomes **a run** inside the trace.
3. You can visualize the path taken:
    - START –> Retriever –> Reranker –> LLM Answer –> END
4. If a workflow branches (conditional/parallel/subgraph), LangSmith captures which path was executed and execute path becomes trace rest path is not included in trace

Other Features of LangSmith

1. **Monitoring and Alerting**
   What it does: Monitoring in LangSmith looks across many traces at once to track the overall health of your LLM system. It aggregates key metrics like **latency (P50, P95, P99), token usage, cost, error rates, and success rates**. You can **set up alerts to notify you when these metrics drift outside acceptable ranges** (e.g., a spike in latency, higher error rates, or unexpected cost growth).
   Why it matters: In production, issues often appear first as patterns across multiple runs rather than in a single trace. Monitoring helps you catch these early signals before they impact users at scale. Instead of waiting for customer complaints, you're proactively alerted when performance degrades or costs spike, enabling faster response and more reliable applications.

2. **Evaluation**
   What it does: Evaluation in LangSmith helps you systematically measure the **quality of your LLM outputs**. You can run tests against gold-standard datasets or apply **custom evaluation metrics such as faithfulness, relevance, or completeness.** LangSmith supports multiple approaches: automated scoring with **LLM-as-a-judge, semantic similarity checks, or even custom Python evaluators**. Evaluations can be run both offline (batch tests before deployment) and online (continuous checks on live traffic).
   Why it matters: LLM behaviour can be unpredictable—a small change in prompts, models, or retrieval logic may improve some cases but break others. Evaluation provides an objective, repeatable way to track performance over time, ensuring that new versions are actually better and preventing regressions.

Example: For a RAG chatbot, you might evaluate:

- Faithfulness Are answers grounded in retrieved documents?
- Relevance –> Did the response actually address the user's question?
- By running the same dataset across GPT-4, Claude, and LLaMA, you can directly compare which model (or pipeline setup) performs best.

3. **Prompt Experimentation**

   What it does: Prompt Experimentation in LangSmith allows you to **systematically test and compare different prompt versions**. You can run A/B tests across prompts on the same dataset, **track their performance against evaluation metrics, and record the outcomes**. Results are stored over time, giving you a clear history of which prompt variations worked best and under what conditions.

4. **Dataset Creation & Annotation**

   What it does:

   - Provides tools to build datasets for evaluation and fine-tuning.
   - Supports manual annotation (e.g., labelling whether an answer is correct).
   - Stores datasets versioned for reuse across projects.

   Why it matters: High-quality datasets are critical for evaluation and feedback loops.

   Example:

   Customer support: Build a dataset of common questions + expected answers.
   Use it to benchmark your RAG agent every time you change retrieval logic.

5. **User Feedback Integration**

   What it does: Let you **capture thumbs up/down, ratings, or structured feedback from users** in production.

   Feedback is logged alongside traces –> tied to the exact prompt, model, and state.

   Supports bulk analysis of what users like/dislike.

6. **Collaboration**

   What it does: Team members can view, share, and comment on traces, datasets, and evaluations.

   Provides a web UI where non-engineers (PMs, QA, annotators) can inspect and annotate runs.

   Enables shared experiment dashboards.